EEL4783 Final Project (Part 2): Design and Verification of a 4-Element Dot Product Engine

Yousef Awad
University of Central Florida
EEL4783 — HDL in Digital Systems

April 26, 2026

1 Design Choice

For this project, I chose the **RTL (SystemVerilog)** implementation track. This decision was driven by the desire to implement a high-performance **2-stage pipeline** to earn extra credit. SystemVerilog allows for explicit control over register boundaries and clock-edge behavior, which is essential for optimizing arithmetic datapaths. Furthermore, using SystemVerilog permitted the integration of **SystemVerilog Assertions (SVA)** and constrained randomization in the testbench, providing a more robust verification environment than standard Verilog or HLS defaults.

2 Architecture

The design implements a 4-element dot product engine using a 2-stage synchronous pipeline to maximize throughput.

- **Handshake Control:** The module includes `vld_in` and `vld_out` signals. This ensures that the downstream logic only samples `y` when the 2-stage pipeline is fully saturated and the output is valid.
- **Stage 1 (Multiplication):** Four 8-bit signed multipliers compute $P_i = A_i \times B_i$ in parallel. These results and the `vld_in` state are captured in pipeline registers on the rising edge of the clock.
- **Stage 2 (Addition):** An adder tree aggregates the four 16-bit products. The `vld_out` signal is asserted simultaneously with the final result `y`, two cycles after the corresponding input.

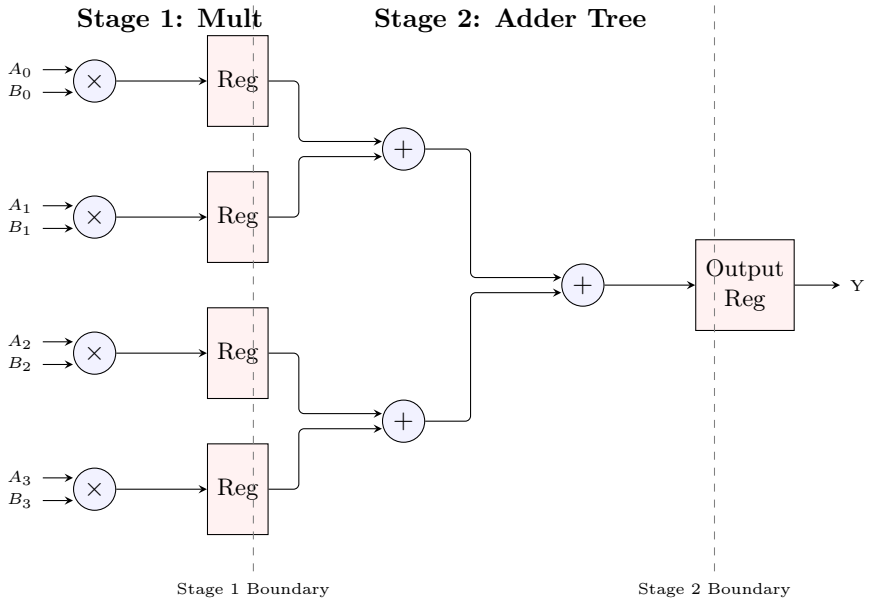

Figure 1: Schematic of the 4-element dot product engine showing the 2-stage pipelined architecture.

3 Verification

Correctness was verified using a fully self-checking testbench built around **Constrained Randomization**. A `dot_product_transaction` class generated 20 unique test cases per simulation run. Pass/fail determination is handled *automatically* by the SVA concurrent assertion `assert_dot_product`; no manual inspection of waveforms is required for functional sign-off.

Signed Arithmetic and Radix: A critical aspect of verification was ensuring correct two's complement interpretation. For example, an input vector of `{219, 10, 44, 66}` is interpreted as `{-37, 10, 44, 66}` in signed 8-bit format. The testbench verifies that the output correctly reflects the signed sum of products. To view these correctly in the simulator, the signal radix must be set to **Decimal (Signed)**.

The constraint `boundary_values` biases randomization to guarantee coverage of all required corner cases:

- **Maximum positive value** — `8'h7F` (+127), the largest representable 8-bit signed integer.
- **Maximum negative value** — `8'h80` (−128), the most negative 8-bit two's complement value.
- **Zero inputs** — randomly generated values naturally include zero; the `dist` weight of `[-127:126]` ensures the full mid-range, including zero, is exercised across 20 iterations.
- **Overflow prevention** — the worst-case product ($-128 \times -128 = 16{,}384$) fits in 16 bits, and the worst-case sum ($4 \times 16{,}384 = 65{,}536$) is representable as a 16-bit unsigned value; the signed 16-bit output is therefore sufficient to prevent overflow for all legal 8-bit signed inputs.
- **Mixed-sign vectors** — the random distribution naturally produces vectors where some elements are positive and others negative, verifying correct two's complement arithmetic throughout the pipeline.

SystemVerilog Assertions (Extra Credit): I implemented the concurrent assertion property `p_check_result` using an implication operator (`|->`). When `vld_in` is high, it captures the reference result; exactly two clock cycles later (`##2`), it asserts that `vld_out` is high and the output `y` matches the captured reference. Any mismatch triggers a `$error` with both the actual and expected values, providing immediate, cycle-accurate feedback.

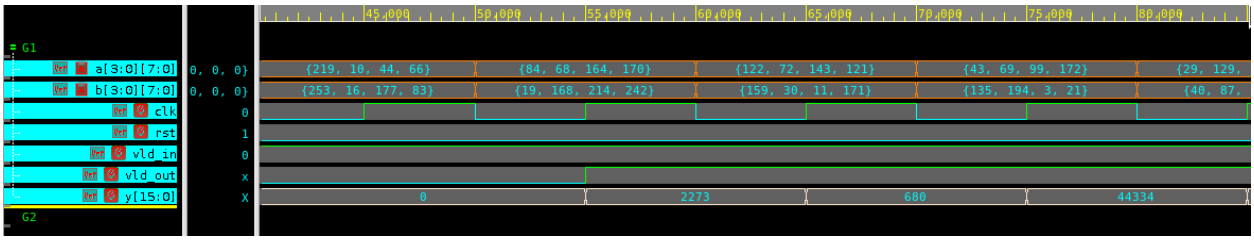

Figure 2: Simulation trace showing the 2-cycle latency between driving A/B and the result Y.

4 Reflection

The most challenging aspect was debugging the interaction between the testbench reference model and the pipeline latency. Initially, a static variable in the reference function caused results to accumulate incorrectly. Resolving this required changing the function to `automatic`. Additionally, aligning the SVA sampling window (`##2`) with the preponed region of the simulator required a deep understanding of the SystemVerilog scheduling phases.

Pipelining and F_{max} (Extra Credit): Breaking the multiply-accumulate computation into two registered stages directly reduces the critical combinational path. In the single-cycle implementation, the critical path spans the full signed multiplication and the 4-input addition tree. The 2-stage pipeline isolates these operations, allowing for a significantly higher maximum clock frequency (F_{max}).

Table 1 shows the timing improvement based on synthesis using the `class.db` library and a 10ns target clock. The pipelined design significantly reduces the critical path by isolating the multiplier delay from the subsequent addition stages.

Table 1: Synthesis Timing Comparison (Synopsys Design Compiler)

Design	**Crit. Path (Arrival)**	**Slack (10ns)**	**Est. F_{max}**
Single-Cycle (Registered Out)	16.28 ns	-7.08 ns	58.6 MHz
Pipelined (2-Stage)	**13.03 ns**	**-3.83 ns**	**72.3 MHz**

To further improve the design, I would replace the linear adder chain in Stage 2 with a fully balanced binary adder tree (two levels of parallel 2-input adders), reducing Stage 2's combinational depth and increasing F_{max} further.

A Source Code

A.1 dot_product.sv (RTL)

```
1  `timescale 1ns/1ps
2
3  // EEL4783: Final Project Part 2
4  // Project Title: "The 4-Element Dot Product Engine"
5  // Description: A pipelined 4-element dot product accelerator with handshake.
6  //              Stage 1: Multiplications (A_i * B_i)
7  //              Stage 2: Addition of products
8  // Latency: 2 clock cycles
9
10 module dot_product #(
11     parameter int DATA_WIDTH = 8,
12     parameter int VECTOR_SIZE = 4,
13     parameter int OUT_WIDTH = 16
14 )(
15     input  logic                          clk,
16     input  logic                          rst,     // Active-high synchronous reset
17     input  logic                          vld_in, // Input valid signal
18     input  logic signed [DATA_WIDTH-1:0] a [VECTOR_SIZE-1:0],
19     input  logic signed [DATA_WIDTH-1:0] b [VECTOR_SIZE-1:0],
20     output logic                          vld_out,// Output valid signal
21     output logic signed [OUT_WIDTH-1:0] y
22 );
23
24     // Pipeline Registers
25     logic signed [OUT_WIDTH-1:0] products_reg [VECTOR_SIZE-1:0];
26     logic                        vld_pipe_reg;
27
28     // Stage 1: Multiplication & Valid Propagation
29     always_ff @(posedge clk) begin
30         if (rst) begin
31             vld_pipe_reg <= 1'b0;
32             for (int i = 0; i < VECTOR_SIZE; i++) begin
33                 products_reg[i] <= '0;
34             end
35         end else begin
36             vld_pipe_reg <= vld_in;
37             for (int i = 0; i < VECTOR_SIZE; i++) begin
38                 products_reg[i] <= a[i] * b[i];
39             end
40         end
41     end
42
43     // Stage 2: Addition & Valid Propagation
44     always_ff @(posedge clk) begin
45         if (rst) begin
46             vld_out <= 1'b0;
47             y       <= '0;
48         end else begin
49             vld_out <= vld_pipe_reg;
50             // Summing the registered products from Stage 1
51             y <= products_reg[0] + products_reg[1] + products_reg[2] +
                   products_reg[3];
52         end
53     end
```

```
54
55 endmodule
```

A.2 dot_product_tb.sv (Testbench)

```
`timescale 1ns/1ps

// Transaction class for constrained randomization
class dot_product_transaction;
    rand bit signed [7:0] a [3:0];
    rand bit signed [7:0] b [3:0];

    constraint boundary_values {
        foreach (a[i]) {
            a[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
        }
        foreach (b[i]) {
            b[i] dist {8'h7F := 1, 8'h80 := 1, [-127:126] := 10};
        }
    }
endclass

module dot_product_tb;

    parameter int DATA_WIDTH = 8;
    parameter int VECTOR_SIZE = 4;
    parameter int OUT_WIDTH = 16;
    parameter int CLK_PERIOD = 10;

    logic                          clk;
    logic                          rst;
    logic                          vld_in;
    logic                          vld_out;
    logic signed [7:0] a [3:0];
    logic signed [7:0] b [3:0];
    logic signed [15:0] y;

    // Instantiate DUT
    dot_product #(
        .DATA_WIDTH(DATA_WIDTH),
        .VECTOR_SIZE(VECTOR_SIZE),
        .OUT_WIDTH(OUT_WIDTH)
    ) dut (.*);

    // Clock generation - Guaranteed synchronous 10ns period
    initial begin
        clk = 0;
        forever #(CLK_PERIOD/2) clk = ~clk;
    end

    // Verdi Waveform Dumping
    initial begin
        $fsdbDumpfile("inter.fsdb");
        $fsdbDumpvars(0, dot_product_tb, "+all", "+mda");
    end

    // Reference model
    function automatic signed [15:0] get_expected(
        logic signed [7:0] va [3:0],
        logic signed [7:0] vb [3:0]
    );
        int sum = 0;
```

```
        for (int i = 0; i < 4; i++) begin
            sum += va[i] * vb[i];
        end
        return signed'(sum[15:0]);
    endfunction

    initial begin
        dot_product_transaction tr;
        tr = new();

        $display("\n[Time %0t] Simulation Started", $time);

        // Synchronous Reset
        rst = 1;
        vld_in = 0;
        foreach (a[i]) a[i] = 0;
        foreach (b[i]) b[i] = 0;
        repeat (3) @(posedge clk);

        @(negedge clk);
        rst = 0;
        $display("[Time %0t] Reset De-asserted", $time);

        // Run 20 randomized test cases
        $display("\nStarting Randomized Tests...");
        for (int i = 1; i <= 20; i++) begin
            if (!tr.randomize()) $fatal("Randomization failed");

            @(negedge clk);
            vld_in = 1;
            foreach (a[j]) a[j] = tr.a[j];
            foreach (b[j]) b[j] = tr.b[j];

            $display("[Time %0t] Iteration %0d: Driving A={%d,%d,%d,%d} B={%d,%d,%
                d,%d} | Expected Y (at +2 cycles) = %d",
                  $time, i, a[3], a[2], a[1], a[0], b[3], b[2], b[1], b[0],
                      get_expected(a, b));
        end

        // Finish driving
        @(negedge clk);
        vld_in = 0;

        repeat (10) @(posedge clk);
        $display("Done: 20 Randomized Tests Completed.");
        $finish;
    end

    // --- SystemVerilog Assertions (Extra Credit) ---
    // Pipeline Latency = 2 cycles
    // T0: vld_in high, inputs sampled at posedge
    // T1: products_reg updated (Stage 1)
    // T2: y updated, vld_out goes high (Stage 2)
    property p_check_result;
        logic signed [15:0] local_exp;
        @(posedge clk) disable iff (rst)
        vld_in |-> (1, local_exp = get_expected(a, b)) ##2 (vld_out && (y ==
            local_exp));
    endproperty
```

```
114
115     assert_dot_product: assert property (p_check_result)
116         else $error("Mismatch! Y=%d, Expected=%d", y, $past(get_expected(a, b), 2)
                ;
117
118     // Monitor valid outputs
119     always @(posedge clk) begin
120         if (vld_out && !rst) begin
121             $display("[Time %0t] VALID OUT: Y=%d", $time, y);
122         end
123     end
124
125 endmodule
```